The Monday After the 3 AM Page

Lecture 3 · Your IDE as a professional tool

VS Code · Python · debugging · profiling

(No, you can't go back to sleep.)

Previously, on Mastering CS…

- 3 AM. Checkout down. Jordan's note. A rogue process.
- You fixed it with `grep`, pipes, `chmod` and `kill`.
- NovaStack noticed.

> This morning you've been moved onto **the Orders team**: the team that owns the code behind checkout.

Monday, 9:00

> **From:** Jordan · **Subject:** Welcome to Orders 👋
>
> 1. **Set up your laptop the team's way.** Don't improvise.
> 2. **Ticket ORD-317 is yours.** We've been *undercharging loyalty customers* since the last release. Finance is not happy.
> 3. Then: the **month-end sales report** has become painfully slow.
>
> The codebase is big and you didn't write it. That's normal. **Use the tools.**

ORD-317

> **Loyalty customers undercharged on meal deal orders** · P2 · pricing
>
> *"Sam Patel (Silver card) bought a chicken wrap, fries, a cola and an orange juice and paid **£5.95**. By my maths it should be **£7.29**."*
> — Grace Adeyemi, Finance

The codebase

45 modules · ~220 tests

4 developers · 5 months of history

You

Two weeks in

Never seen this code before

What new developers actually struggle with

Finding	Source
New hires struggle most with **large codebases they didn't write**, and the team around them	Begel & Simon, 2008
Most refactorings are done **by hand** (~90%), not with the tools that make them safe	Murphy-Hill et al., 2012
Professionals **under-use debuggers**; few know the advanced features	Beller et al., 2018
84% use AI tools; **46% don't trust** what they produce	Stack Overflow, 2025

Today: the IDE features that close each of these gaps.

Act 1

"What does everyone actually use?"

Development environments

Three kinds of development environment

	Example	Good for
Terminal-based	tmux + Vim + a shell (Jordan at 3 AM)	Servers over SSH, no install, lightweight
Graphical IDE	VS Code: editor, language support, Git, debugger, terminal, AI	Day-to-day development
Containers / cloud	The same VS Code, tools in a Dev Container or Codespaces	Everyone gets the *same* environment

VS Code: **75.9%** of developers (Stack Overflow 2025). Cursor? A VS Code fork.

> **Q:** How do you exit Vim?
>
> **A:** Nobody knows. People just buy a new laptop.

Act 2

"Set up your laptop the team's way"

Python set-up and extensions

"It works on my machine"

> **Developer:** It works on my machine.
>
> **Team lead:** Great. Then we'll ship your machine.
>
> *(…and that's how containers were invented.)*

The professional answer: the set-up **lives in the repository**.

File	Shares
`pyproject.toml` + `uv.lock`	Exact Python packages, for every OS
`.python-version`	The Python version
`.vscode/settings.json`	Team rules: Ruff on save, rulers, line endings
`.vscode/extensions.json`	The extensions VS Code offers
`.devcontainer/`	The whole toolchain, in a container

🖥️ Demo 1, part 1: open it the team's way

1. **Trust** the folder (Workspace Trust / Restricted Mode)
2. **Install** the recommended extensions
3. **Select the interpreter**: the `.venv` (made by `uv sync`)
4. Mess up some spacing → **save** → Ruff fixes it
5. Extensions view: `@recommended` · publishers · Profiles
6. **Reopen in Container**: same tools, Linux, for everyone

> Most "VS Code isn't working" problems are **the wrong interpreter**.

Act 3

"Find your way around code you didn't write"

Language servers

Where the "intelligence" comes from

Before: every editor implements every language

M editors × N languages

With the Language Server Protocol: each editor speaks LSP once; each language has one server

M + N

Python's language server: Pylance

- completion and hover docs
- go to definition, find references
- rename, extract, auto-import
- errors as you type: `Money.of(1.5)` ❌

> Scrolling is not a search strategy.

🖥️ Demo 1, part 2: navigate by meaning

Move	Win/Linux · macOS
Go to Symbol in Workspace	`Ctrl+T` · `Cmd+T`
Go to Definition / Peek	`F12` / `Alt+F12` · `Opt+F12`
Find All References	`Shift+F12`
Go to Implementations	`Ctrl+F12` · `Cmd+F12`

Reproduce: `uv run novastack quote …` → **£5.95**, loyalty discount **×3**

`_apply(self._loyalty, …)` is **inside the category loop**

GitLens: *Priya, 10 Sep: "category subtotals and loyalty in one pass (ORD-301)"*

> `git blame` is for **context**, not for blaming. (Priya is on holiday. Leave her alone.)

Act 4

"Leave it better than you found it"

Refactoring

Refactoring: structure changes, behaviour doesn't

> There are only two hard things in computer science: cache invalidation, naming things, and off-by-one errors.

> **Mei:** *Please* rename `calc`. Nobody knows what it calculates.

Find & Replace	Rename Symbol (F2)
Changes text	Changes **one symbol**
Hits `calculator`, comments, strings	Hits definition, imports, uses
Hope	Preview, then apply

~90% of refactorings are done by hand (Murphy-Hill et al., 2012). Don't be the 90%.

Act 5

"Get a pair programmer"

AI-powered development

Three ways to work with AI

Mode	How	Good for
Autocomplete	Type; `Tab` accepts	Finishing what you started: steer with a **good name + docstring**
Inline chat	Select, `Ctrl/Cmd+I`	Changing a specific piece; review the **diff**
Chat / agent	Chat view: Ask, Plan, Agent	Questions about the codebase; multi-file tasks

`AGENTS.md` = the team's rules for the AI · `chat.useAgentsMdFile: true` · **never paste secrets**

> **You:** Why are loyalty customers undercharged?
>
> **AI:** Great question! *(sounds very sure)*
>
> **Jordan:** Great. Now **prove it**.

What the studies say about AI and speed

Study	Finding
Cui et al., *Management Science* 2026 (4,867 developers)	**+26%** tasks completed; juniors gain most
METR randomised trial 2025 (preprint)	Experienced devs **19% slower**… but *felt* **20% faster**
Stack Overflow 2025	Top frustration (66%): "almost right, but not quite"

> Felt 20% faster, was 19% slower. Like me at the gym.

AI is a lead, not an oracle. Read it. Run it. Prove it.

Act 6

"Why are loyalty customers undercharged?"

Debugging

The stages of print debugging

```
print("here")
print("here 2")
print("HERE??")
print(running)            # Money(amount=Decimal('9.50'), currency='GBP') ...ok?
print("why is this happening")
```

> "*The most effective debugging tool is still careful thought, coupled with judiciously placed print statements.*" — Brian Kernighan

Today: what comes after the print statements.

Reproduce → hypothesise → observe → fix → verify

🖥️ Demo 3: from logs to proof

1. **Logs first:** `NOVASTACK_LOG_LEVEL=DEBUG` → `applied LOYALTY` **three times**
2. **Conditional breakpoint** `category == "drink"`, run with **F5** (CLI launch config)
3. **Variables / Watch / Call Stack**: running **£9.50**, already **2** discounts
4. **Debug Console**: `[d.code for d in applied]`
5. **Logpoint**: a `print()` you never have to delete
6. **`breakpoint()` / pdb**: for when there's no IDE (servers, 3 AM)
7. **AI from evidence**: paste the log, ask why
8. **Fix → £7.29** ✅

Fixed! …and a test failed?

```
$ uv run pytest -m smoke
FAILED tests/unit/pricing/test_calculator.py::test_loyalty_discount_applied_once
[XPASS(strict)] ORD-317: remove when fixed
1 failed, 17 passed
```

> We fixed the bug and a test failed. Welcome to software engineering.

Hold that thought: **Lecture 4**.

Act 7

"Why is the month-end report so slow?"

Profiling

Measure first

> **Grace:** The month-end report used to take a couple of seconds. Now it takes ages. Board meeting on Friday. Help?

> "*Premature optimisation is the root of all evil.*" — Donald Knuth

> **Jordan:** Measure first. Programmers are terrible at guessing where the time goes.

Tool	Question it answers
`time`	Computing or waiting? (real vs user vs sys)
`cProfile`	Which functions, how many calls?
pyinstrument, flame graphs	Where is the time, as a picture?
`hyperfine`	Is it *really* faster, or was that a fluke?

🖥️ Demo 4: two hot spots

Step	What we saw
`time`	~6 s, almost all CPU; suspicious **1 s of sys**
`cProfile`	a generator called **70,820,463** times
pyinstrument	`is_returning_customer` ≈ ⅔ · `category_of` ≈ ⅓
Fix 1: one pass, not a scan per order	**~3.5 s**
Re-profile → re-reading the menu **38,824** times	
Fix 2: read it once	**~0.5 s** ✅ same output

> O(n²): twice the orders, **four times** the work in that function. That's what quadratic feels like.

> **Grace:** 0.5 seconds?! I'll take it. 🎉

Beyond the IDE: the wider toolbox

Tool	For
`htop` · Activity Monitor · Task Manager	What's using CPU, memory, disk *right now*
`py-spy`	Profiling an already-running Python process
`tracemalloc` · `memory-profiler`	Where memory goes
`perf` + flame graphs	Native code (C, C++, Rust)
`strace` · `bpftrace` · `tcpdump`	What a program asks the OS, or sends on the network

Friday, 16:30

> **From:** Jordan
>
> Brilliant first week: ORD-317 fixed, the report flying, `calc` finally has a name. 🎉
>
> One thing: all of it is still **on your laptop**. On this team, nothing reaches customers without **tests**, a **review** and **CI**. Dan's question stands: *how do we make sure nobody breaks it again?*
>
> That's next week.

Lecture 4: CI/CD: tests, Git and pull requests, code review, GitHub Actions.

Before next week

- **Exercise ORD-318:** the student discount vanishes on Fridays (debugger only, no `print()`!)
- **Labs**, pick one: **Beginner** (do the lecture yourself) · **Intermediate** (new bugs, fewer hints) · **Advanced** (the auditors are coming); each has a self-check
- Re-do any demo: every step is written up in `lectures/03-…/demo/`
- Further reading: the recommended reading on Moodle

Exit ticket (Mentimeter):

1. One shortcut you'll use tomorrow
2. What did the debugger show that `print()` wouldn't?

Thank you

Questions?

Remember: `print("here")` is a cry for help. Use a breakpoint.